ICDFA NIGERIA

Ethical Hacking Internship | Practical Laboratory Assessment

SOC TECHNICAL LABORATORY REPORT

Centralised Endpoint Security Monitoring with Wazuh SIEM

Real-time File Integrity Monitoring of a Windows endpoint: deployment, detection and alert analysis

Student details	
Name	Abubakar Mahmood Muhammad
Registration number	C11/26/EHIT/17332
Programme	Ethical Hacking Internship, ICDFA Nigeria
Role in scenario	Level 1 Security Operations Center (SOC) Analyst
Platform	Wazuh 4.12.0 (Manager, Indexer, Dashboard, Windows agent)
Date of practical	16 October 2026
Date of report	17 October 2026

All screenshots and alert extracts in this report were captured by the author during the practical. The four FIM alert records are also supplied unmodified as JSON files in the accompanying repository.

Contents

1. Introduction
2. Objectives
3. Laboratory architecture and environment
4. Phase 1: Wazuh SIEM deployment and verification
5. Phase 2: Wazuh Dashboard access
6. Phase 3: Windows agent installation and registration
7. Phase 4: File Integrity Monitoring configuration
8. Phase 5: Security event generation
9. Phase 6: Alert investigation and findings
10. Responses to the four laboratory questions
11. Technical challenges and resolutions
12. Conclusion and recommendations
13. References

Appendix A: Evidence checklist

Appendix B: Command reference

1. Introduction

The Chief Information Security Officer (CISO) of the organisation in this scenario is concerned about unauthorised changes to sensitive files on employee workstations and about the organisation's limited ability to detect and investigate them. As the Level 1 SOC analyst assigned to the problem, my task was to design, deploy, configure and validate a centralised monitoring environment built on Wazuh, an open-source SIEM and endpoint security platform.

Wazuh was used as the central platform: a Wazuh Manager, Indexer and Dashboard on an Ubuntu server, and a Wazuh agent on a Windows endpoint. File Integrity Monitoring (FIM) was configured in real-time mode for a `C:\SensitiveFiles` directory. A test file was then created, modified and deleted, and the resulting alerts were investigated.

The report follows the six compulsory phases of the brief. Every result quoted here comes from the practical performed on 16 October 2026, and alert values are copied from the alert records in the Wazuh index (supplied in the repository under `alerts/`).

2. Objectives

1. Deploy and configure the Wazuh Manager, Indexer and Dashboard on an Ubuntu virtual machine, and verify that the components run.
2. Establish network connectivity between the Ubuntu Wazuh server and a Windows endpoint.

3. Install, authenticate and register a Windows Wazuh agent with the central manager.
4. Verify endpoint connectivity and operational status from the Wazuh Dashboard.
5. Configure real-time FIM for a designated sensitive directory.
6. Simulate controlled file creation, modification and deletion events.
7. Identify, filter and investigate the resulting FIM alerts, and interpret their metadata (timestamps, paths, event types, integrity indicators).
8. Evaluate false positives, ransomware-related use of FIM, and the advantages of centralised SIEM monitoring over native Windows Event Logs.
9. Produce a structured SOC technical report with supporting evidence.

3. Laboratory architecture and environment

The lab used two systems on one VMware NAT network (192.168.48.0/24): the Wazuh server virtual machine and the Windows 11 host acting as the monitored endpoint. The analyst browser ran on the same host and reached the Dashboard over HTTPS.

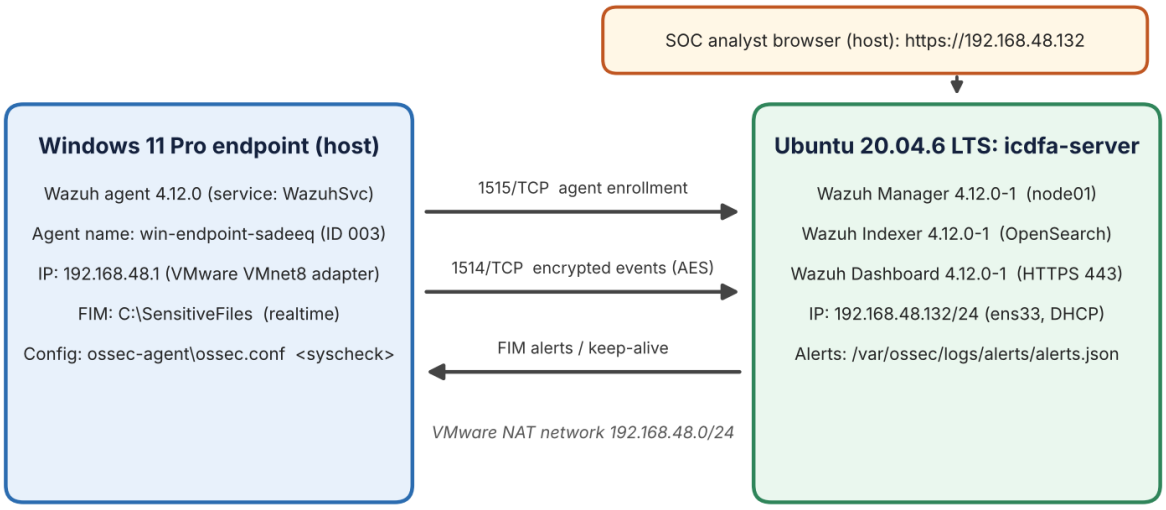


Figure 1: Laboratory architecture and data flow

Item	Detail
Hypervisor	VMware Workstation 17 on a Windows 11 host with 8 GB RAM
Wazuh server VM	`icdfa-server`, Ubuntu 20.04.6 LTS (kernel 5.4.0-216-generic), the laboratory VM supplied by ICDFA
Server software	wazuh-manager, wazuh-indexer and wazuh-dashboard, all version 4.12.0-1
Server network	Interface `ens33`, DHCP, address 192.168.48.132/24 (VMware NAT)
Windows endpoint	Host machine, Windows 11 Pro 10.0.26300.9711, administrator access

Item	Detail
Endpoint address seen by the manager	192.168.48.1 (the host's VMware VMnet8 adapter)
Agent	Wazuh agent 4.12.0-1 (MSI), agent name `win-endpoint-sadeeq`, ID 003, group `default`
Monitored directory	`C:\SensitiveFiles` (real-time FIM)
Network ports	1514/TCP agent events (seen in the agent log); 1515/TCP enrolment (Wazuh default); 443/TCP Dashboard

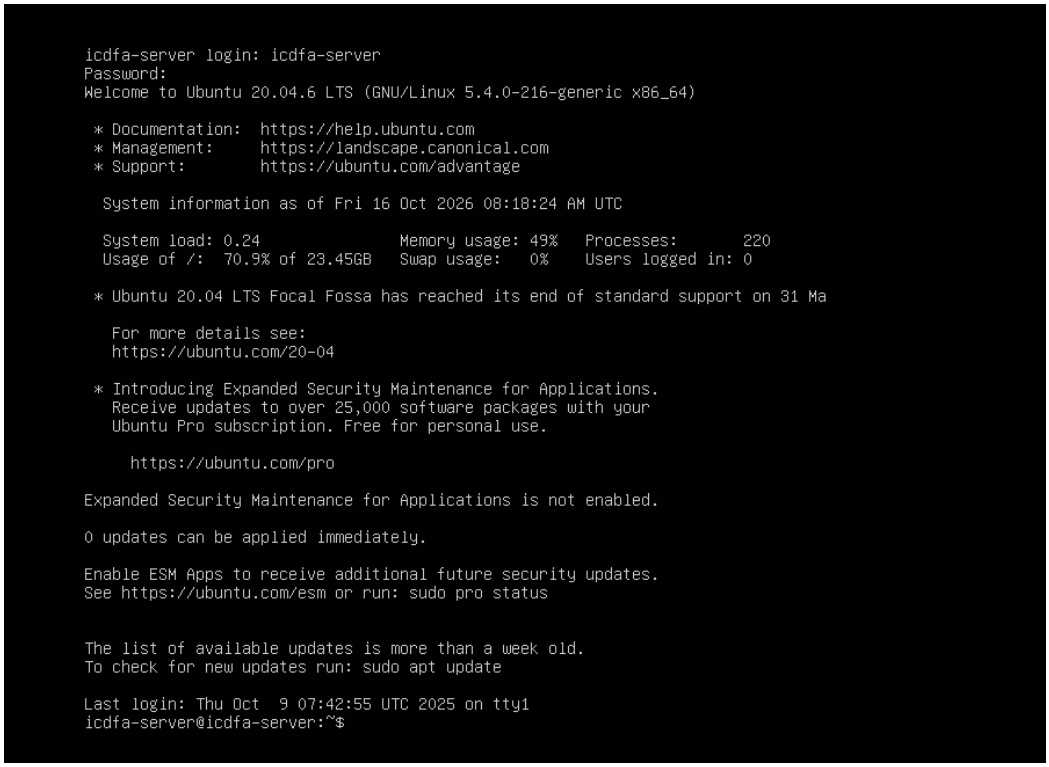


Figure 2: Server login banner: hostname icdfa-server, Ubuntu 20.04.6 LTS, kernel 5.4.0-216-generic

> **Environment note.** The Wazuh stack in this report ran on the ICDFA-provided laboratory VM, which already had the three server components installed. I verified the installation in Phase 1 but did not run the Wazuh installer on that VM. Before using it, I tried to build my own Ubuntu Server 22.04.5 VM in VMware; I abandoned that VM, before installing any Wazuh software on it, after I forgot its login password (Section 11).

4. Phase 1: Wazuh SIEM deployment and verification

4.1 Deployment approach

Wazuh's assisted all-in-one installation places the Manager, Indexer and Dashboard on one server. For reference, the standard command sequence is shown below. It was not executed on the ICDFA VM, because the packages were already present there.

```
curl -sO https://packages.wazuh.com/4.12/wazuh-install.sh
sudo bash wazuh-install.sh -a
```

4.2 Verification of the installed components

Installed packages were listed with `dpkg -l | grep wazuh`. The status flag `ii` means each package is installed and configured. All three components are version 4.12.0-1.

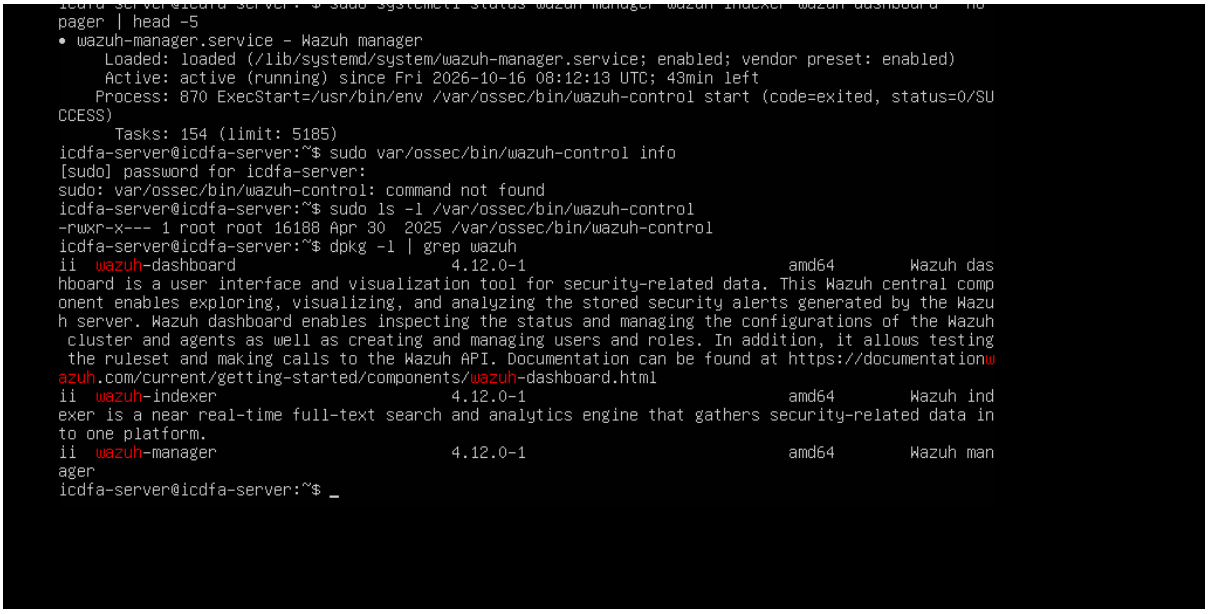


Figure 3: Installed Wazuh packages on the Ubuntu server (dpkg): dashboard, indexer and manager, all 4.12.0-1

Service state was checked with `sudo systemctl status wazuh-manager wazuh-indexer wazuh-dashboard`. The Wazuh Manager and the Wazuh Dashboard show `active (running)`, and the journal records `Started wazuh-indexer` at 08:14:10 UTC.

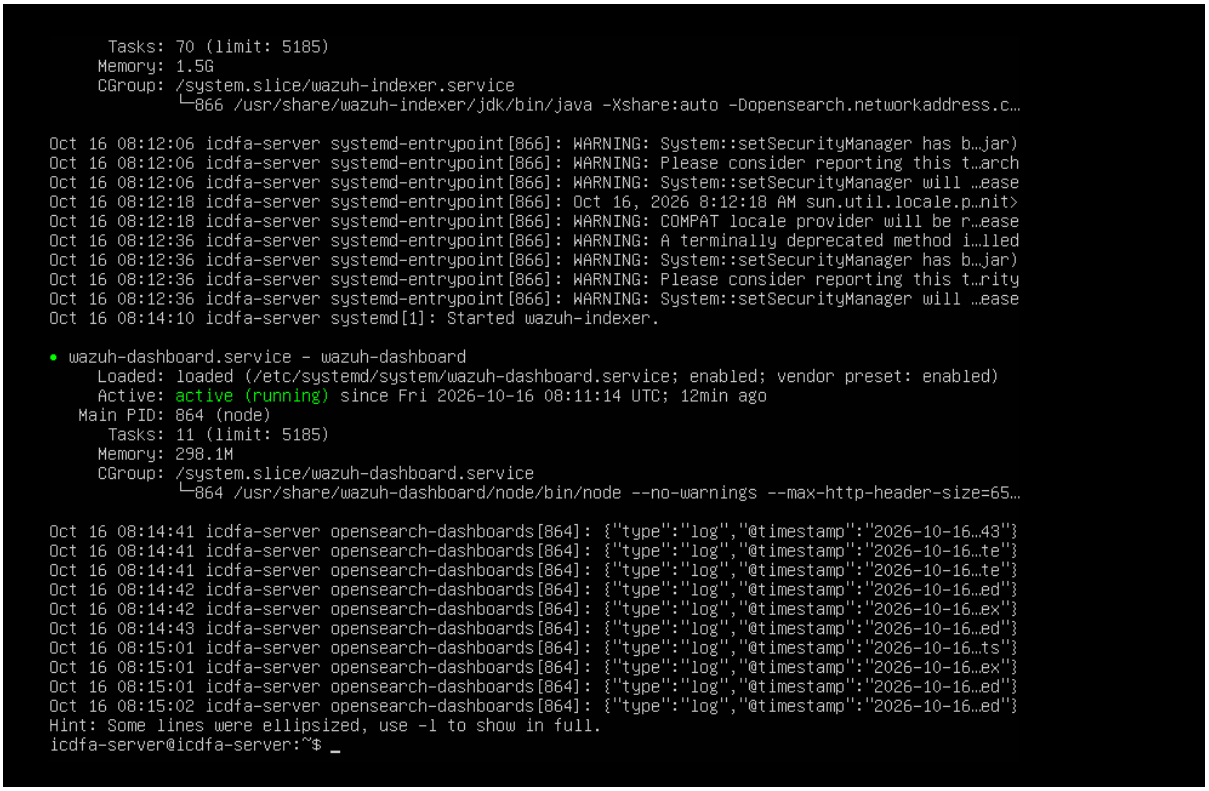


Figure 4: wazuh-indexer journal ("Started wazuh-indexer") and wazuh-dashboard active (running)

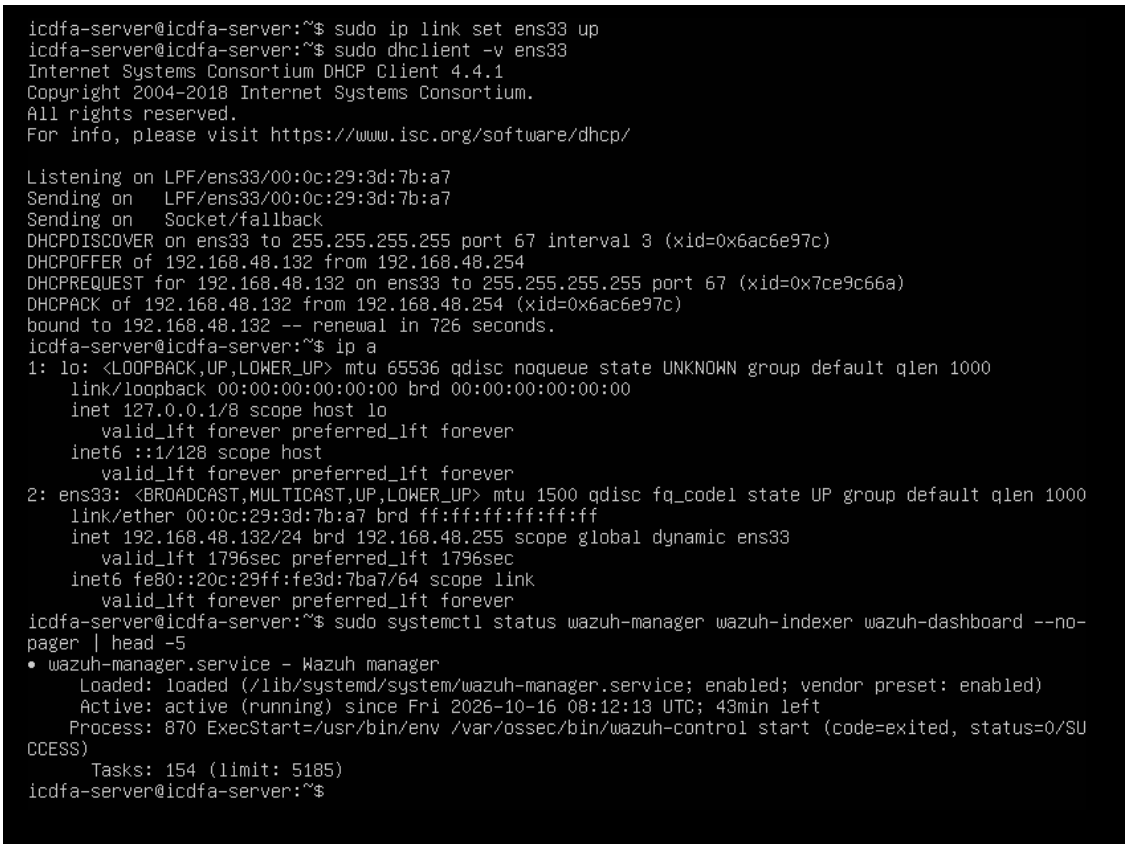


Figure 5: wazuh-manager active (running) since 2026-10-16 08:12:13 UTC, with the address 192.168.48.132 held on ens33

> **Reading the evidence.** Figure 4 and Figure 5 together show the manager, the indexer and the dashboard started within the same boot (08:11 to 08:14 UTC). Figure 3 shows they are installed at version 4.12.0-1. This is the evidence for the "successful installation of Wazuh components" requirement.

5. Phase 2: Wazuh Dashboard access

From the host browser I opened `https://192.168.48.132` and signed in as `admin`. The browser flags the page as "Not secure" because Wazuh ships with a self-signed certificate, which is expected in a lab. The server first had to be given a network address (Section 11, challenge 4): `ip a` then showed `ens33` UP with 192.168.48.132/24.

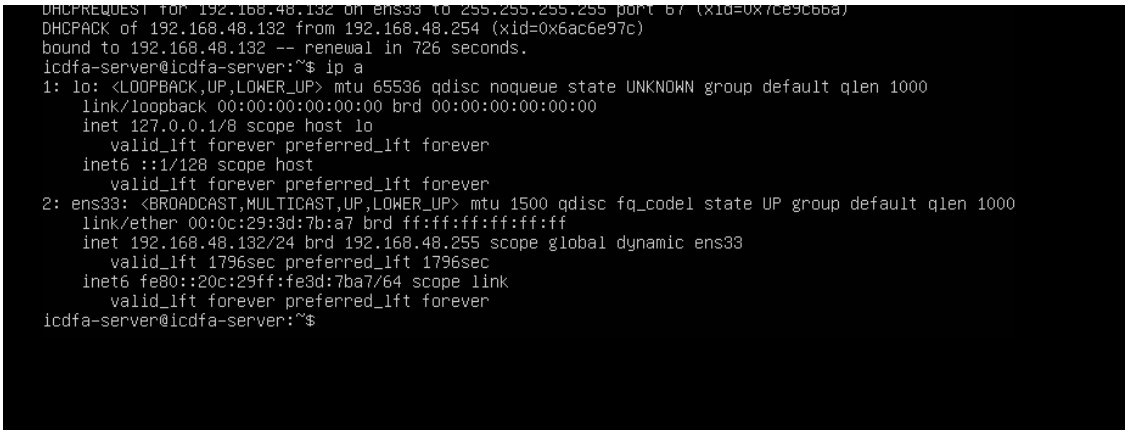


Figure 6: ens33 UP with the DHCP address 192.168.48.132/24

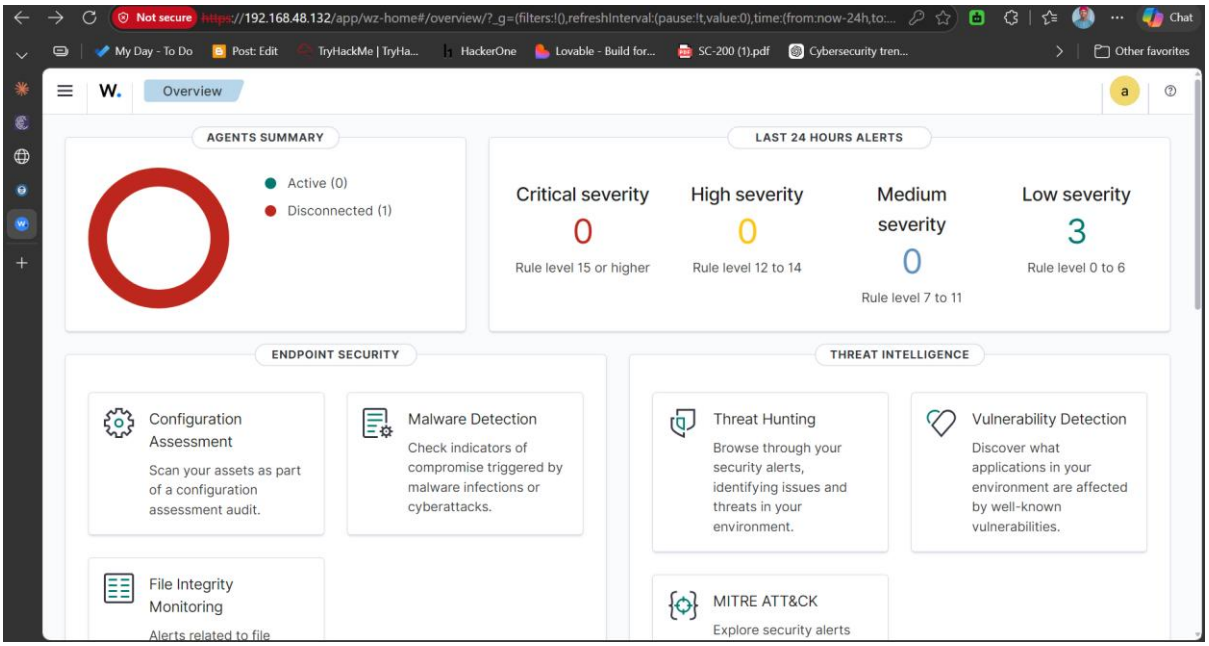


Figure 7: Successful login: Wazuh Dashboard overview at https://192.168.48.132 (Agents summary and Last 24 hours alerts)

The overview page loaded all modules (Configuration Assessment, Malware Detection, File Integrity Monitoring, Threat Hunting, Vulnerability Detection, MITRE ATT&CK). At this point the Agents summary showed no active agent, which is correct because my agent was not installed yet.

6. Phase 3: Windows agent installation and registration

6.1 Installation

The deployment command was generated by the Dashboard wizard (Windows, server address 192.168.48.132, agent name `win-endpoint-sadeeq`) and run in an Administrator PowerShell on the Windows 11 host. It downloads agent 4.12.0-1 and installs it silently.

```
Invoke-WebRequest -Uri https://packages.wazuh.com/4.x/windows/wazuh-agent-4.12.0-1.msi `
  -OutFile $env:tmp\wazuh-agent; msiexec.exe /i $env:tmp\wazuh-agent /q `
  WAZUH_MANAGER='192.168.48.132' WAZUH_AGENT_NAME='win-endpoint-sadeeq'
```

Because `/q` makes `msiexec` return immediately, my first `NET START WazuhSvc` ran before the installer had finished and reported "The service name is invalid" (Figure 8). After the installation completed, the same command started the service (Figure 9).



Figure 8: Agent install command, followed by NET START WazuhSvc run too early ("The service name is invalid")

```
More help is available by typing NET HELPMSG 2185.

PS C:\WINDOWS\system32> NET START WazuhSvc
The Wazuh service is starting.
The Wazuh service was started successfully.

PS C:\WINDOWS\system32>
```

Figure 9: NET START WazuhSvc: "The Wazuh service was started successfully"

6.2 Registration and status

The agent appeared in **Endpoints** as ID 003 with status **active**, registered on 16 Oct 2026 at 09:12:20 and last seen (keep-alive) at 09:13:52.

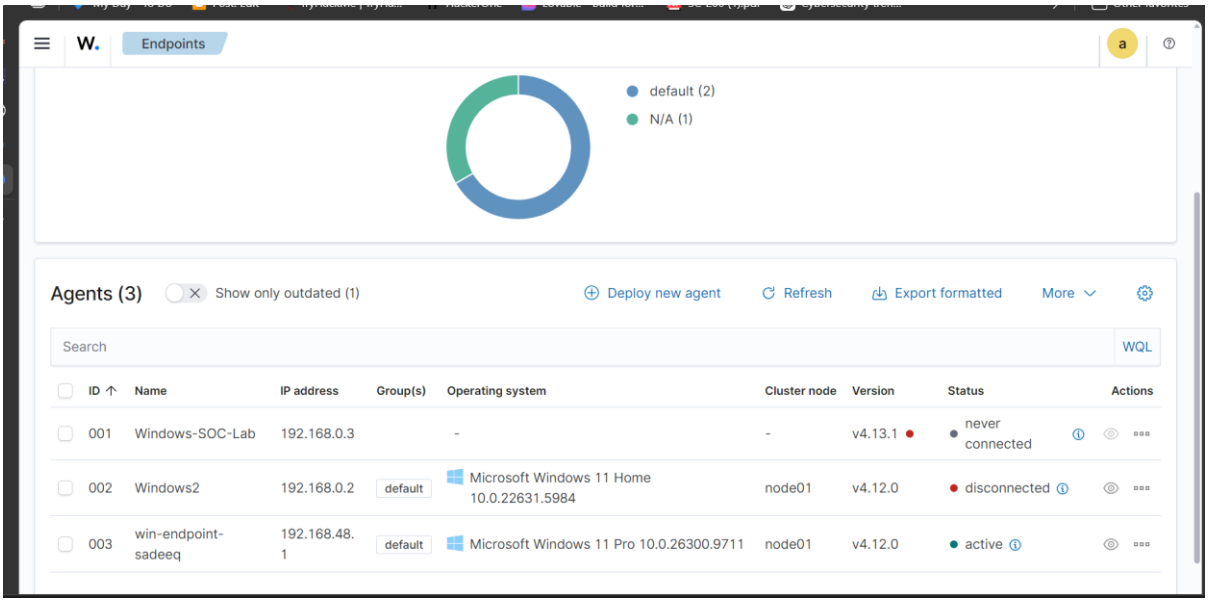


Figure 10: Endpoints list: win-endpoint-sadeeq (ID 003) is active, version 4.12.0, Windows 11 Pro

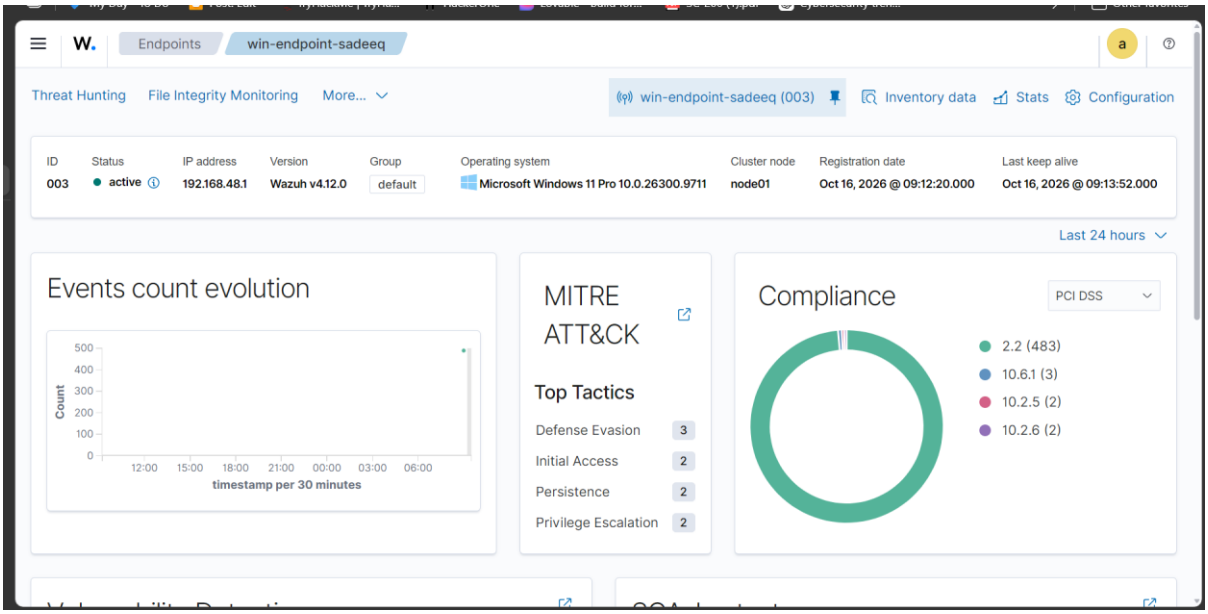


Figure 11: Agent 003 detail page: active, 192.168.48.1, Wazuh v4.12.0, cluster node01, group default

Attribute	Value
Agent ID / name	003 / win-endpoint-sadeeq
Status	active
IP address	192.168.48.1
Version	Wazuh v4.12.0 (same as the manager)
Operating system	Microsoft Windows 11 Pro 10.0.26300.9711
Group / cluster node	default / node01
Registration date	Oct 16, 2026 @ 09:12:20
Last keep alive	Oct 16, 2026 @ 09:13:52

The same list also shows agents 001 (`Windows-SOC-Lab`, never connected) and 002 (`Windows2`, disconnected, with scan data from October 2025). These were already registered on the ICDFA server and are not part of my work. I identified my agent by its name and ID, and all evidence in this report refers to agent 003 only.

6.3 Connectivity verification

The agent log confirms the connection to the manager: `Trying to connect to server ([192.168.48.132]:1514/tcp)`, then `Connected to the server`, then `Agent is now online`.

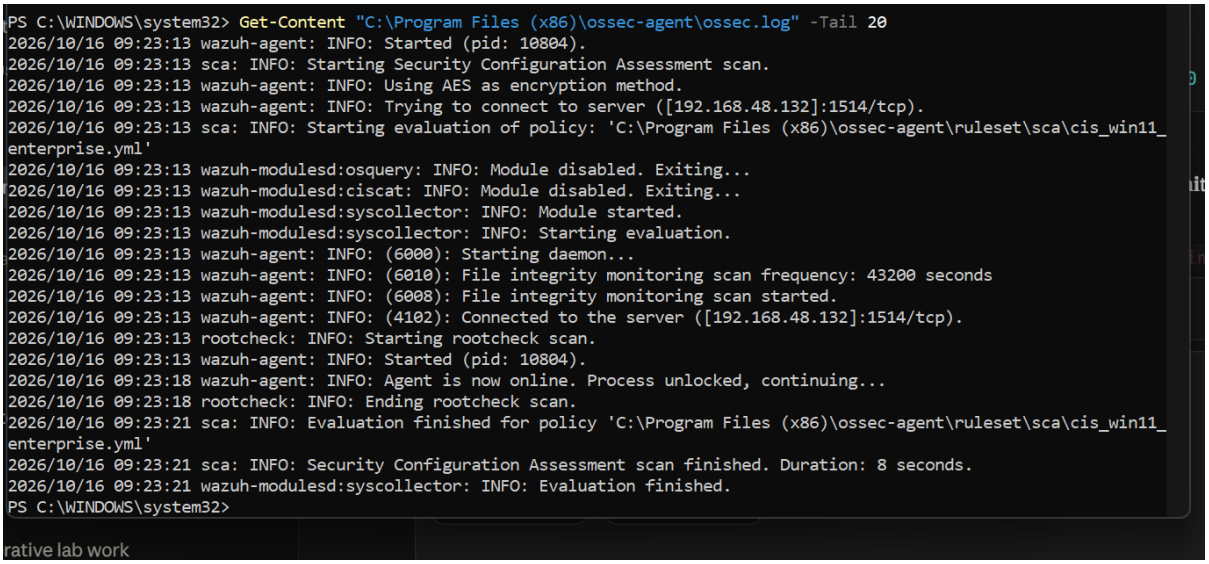


Figure 12: ossec.log: agent connected to 192.168.48.132:1514/tcp and is now online (09:23:13 to 09:23:18)

7. Phase 4: File Integrity Monitoring configuration

The directory `C:\SensitiveFiles` was created on the endpoint, the agent configuration was backed up, and a `<directories>` entry was added inside the `<syscheck>` block of `C:\Program Files (x86)\ossec-agent\ossec.conf`, directly below the `<frequency>` line.

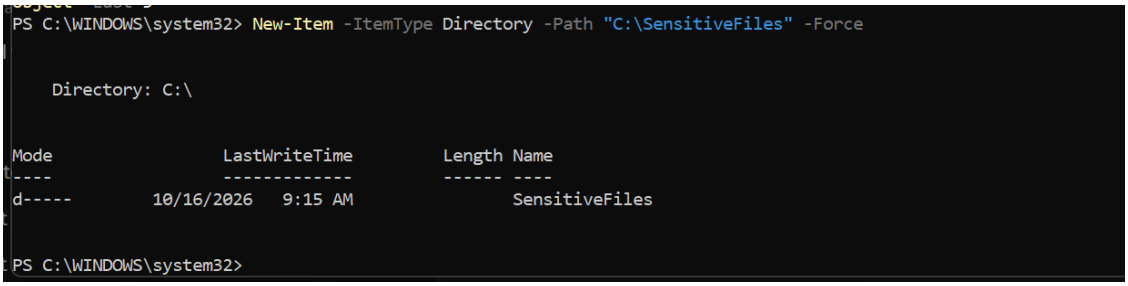


Figure 13: C:\SensitiveFiles created (PowerShell New-Item)

```
<directories realtime="yes" check_all="yes"
report_changes="yes">C:\SensitiveFiles</directories>
```

Attribute	Effect
realtime="yes"	Changes are reported as they happen (Windows change notification) instead of waiting for the next scheduled scan. The default scan interval in this configuration is 43200 s (12 hours).
check_all="yes"	Checks every attribute: size, permissions, owner, group, modification time, inode, MD5, SHA-1 and SHA-256 hashes.
report_changes="yes"	Stores the previous content of text files so the alert can show a diff of what changed.

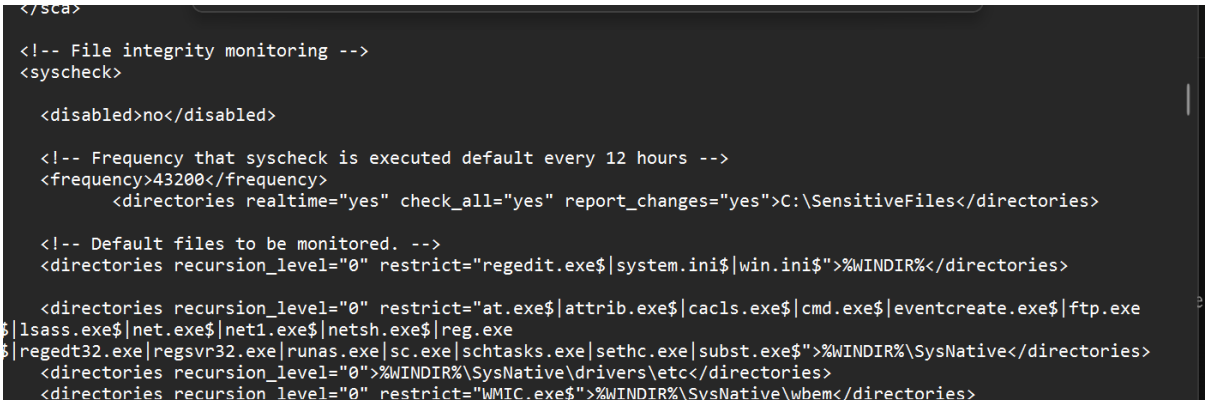


Figure 14: ossec.conf: new real-time directory entry inside <syscheck>, directly below <frequency>

After saving, the agent was restarted (`Restart-Service WazuhSvc`). The agent log then confirmed the configuration was applied: `Monitoring path: 'c:\sensitivefiles'` with the options `size | permissions | owner | group | mtime | inode | hash_md5 | hash_sha1 | hash_sha256 | attributes | report_changes | realtime`. The message appears at 09:20:16 and again at 09:23:13, after the explicit restart.

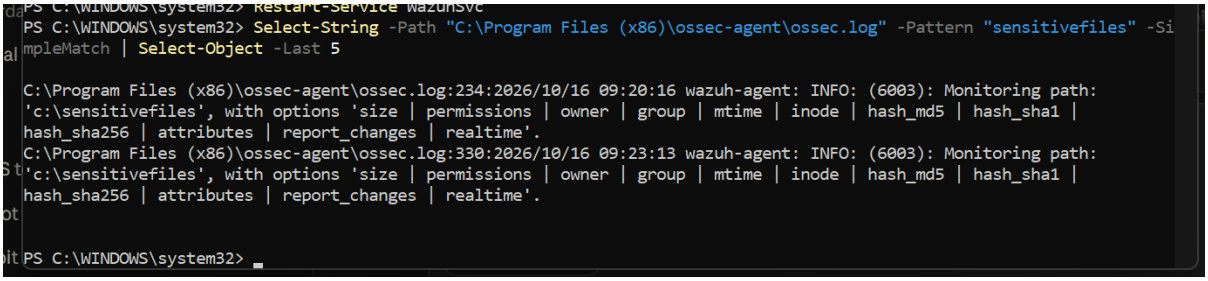


Figure 15: ossec.log: c:\sensitivefiles is monitored with report_changes and realtime

The baseline scan finished at 09:25:27 and real-time monitoring was reported as started at 09:25:33. I waited for this before running any tests, so that the test events were not mixed with the baseline scan.

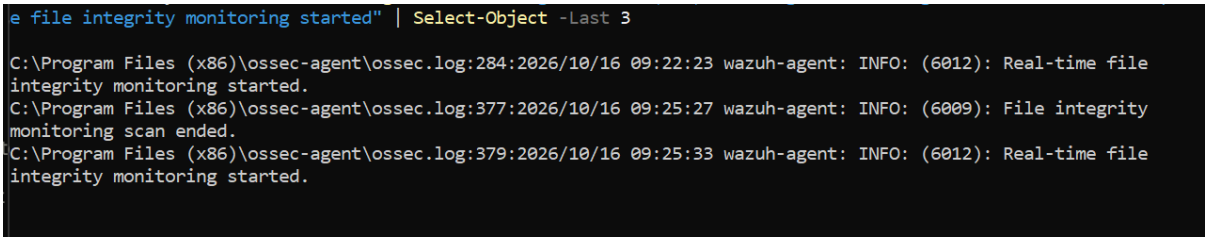


Figure 16: ossec.log: FIM scan ended (09:25:27) and real-time FIM started (09:25:33)

8. Phase 5: Security event generation

Three controlled actions were carried out in an Administrator PowerShell, and `Get-Date` printed the local time (Africa/Lagos, UTC+01:00) before each one.

#	Action	Command	Local time
1	Create	`Set-Content -Path C:\SensitiveFiles\confidential_report.txt -Value "Initial confidential content - SOC lab test"`	09:26:32
2	Create command repeated by mistake (overwrite, same content)	same `Set-Content` command as step 1	09:27:00
3	Modify	`Add-Content -Path ...confidential_report.txt -Value "Unauthorized modification during SOC test"`	09:27:12
4	Delete	`Remove-Item C:\SensitiveFiles\confidential_report.txt`	09:27:31

> **Honest note.** The create command was executed twice (steps 1 and 2). I did not hide this: it produced one extra alert, which is analysed in Section 9 and turned out to be a useful observation about mtime-only changes.

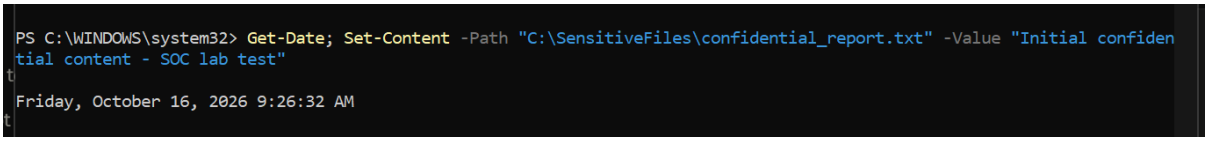


Figure 17: Step 1: file created at 09:26:32

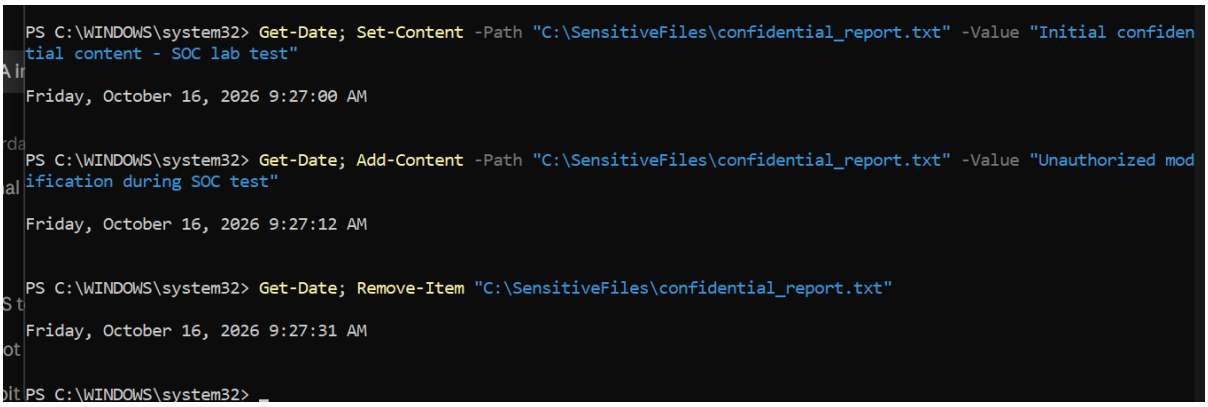


Figure 18: Steps 2 to 4: overwrite 09:27:00, append 09:27:12, delete 09:27:31

9. Phase 6: Alert investigation and findings

9.1 Locating the alerts

The alerts were found in two places: the **FIM: Recent events** panel of the agent's module, and **Discover** (index `wazuh-alerts-*`, time range Last 24 hours, query `confidential_report`). Discover returned exactly **4 hits**, all with `syscheck.mode: realtime`.

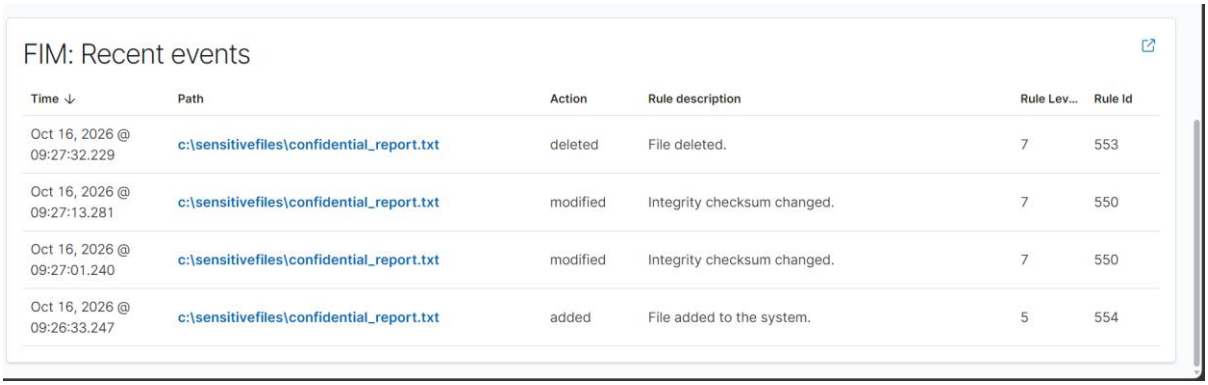


Figure 19: FIM: Recent events: added (554), modified (550) twice, deleted (553) for c:\sensitivefiles\confidential_report.txt

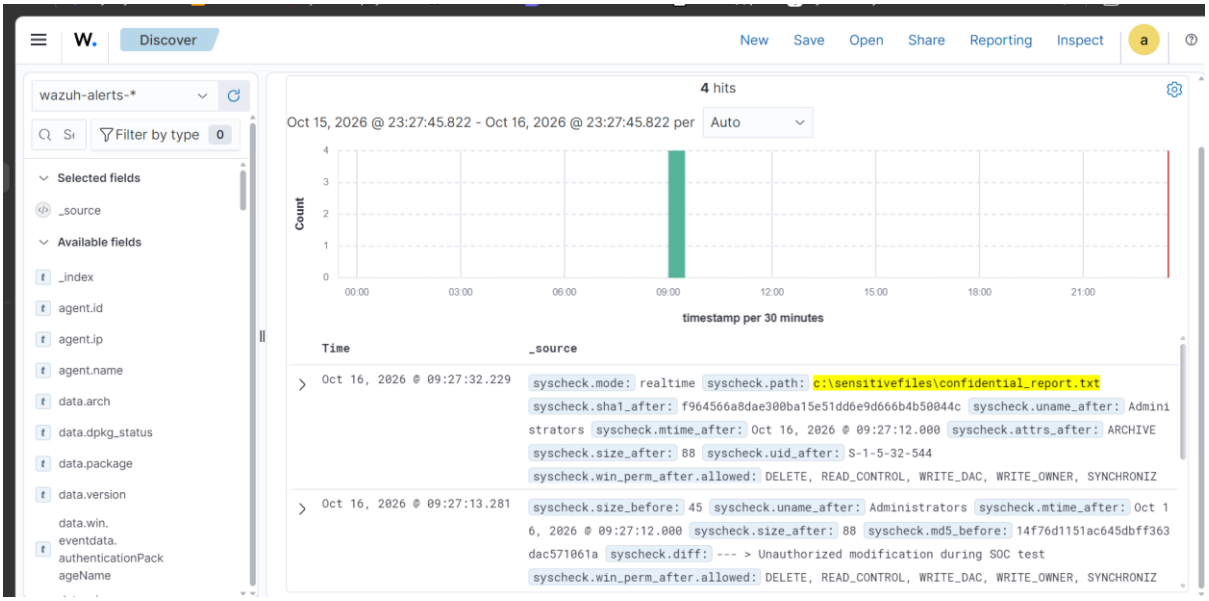


Figure 20: Discover: 4 hits for the test file, syscheck.mode realtime; the modified event shows size 45 to 88 and the diff line

9.2 Alert summary

Dashboard times are Lagos time (UTC+1). The raw `timestamp` field in the JSON records is UTC, one hour earlier.

Time (Lagos)	Event	Rule	Level	Meaning
09:26:33.247	added	554	5	File added to the system
09:27:01.240	modified	550	7	Integrity checksum changed (mtime only)
09:27:13.281	modified	550	7	Integrity checksum changed (content)
09:27:32.229	deleted	553	7	File deleted

Each alert arrived about one second after the PowerShell action, which confirms that real-time monitoring is working (a scheduled scan would have taken up to 12 hours).

9.3 Alert details

Alert 1: file added (rule 554, level 5)

Path `c:\sensitivefiles\confidential_report.txt`, size 45 bytes, mtime 09:26:32, owner `Administrators`. Baseline hashes recorded: MD5 `14f76d1151ac645dbff363dac571061a`, SHA-256 `39c336d735a7d76dc272d017e7542be410b294345927fc6c821c9a4e2b5cb280`.

Alert 2: modified, mtime only (rule 550, level 7)

`changed_attributes` contains only `mtime` (08:26:32 to 08:27:00 UTC). Size and all hashes are identical to Alert 1, and the diff reads "No content changes were found for this file." This is the accidental second create command: Windows rewrote the file with identical content, which still updates its modification time.

Alert 3: modified, content change (rule 550, level 7)

Attribute	Before	After
Size	45 bytes	88 bytes
MD5	14f76d1151ac645dbff363dac571061a	f27fb71cee2eaf3cdf2cbe8adb6b7edd
SHA-1	78f7d7483004f738ca0c1bc94d22d0060e886169	f964566a8dae300ba15e51dd6e9d666b4b50044c
SHA-256	39c336d735a7d76dc272d017e7542be410b294345927fc6c821c9a4e2b5cb280	378189c1375853cbfdf6da4f08b2646a7c6576894235da016369bbb7a9831884
mtime (UTC)	2026-10-16 08:27:00	2026-10-16 08:27:12
Changed attributes	size, mtime, md5, sha1, sha256	
Diff	> Unauthorized modification during SOC test	

The size grew by 43 bytes, which is the appended 41-character line plus a carriage return and line feed. This is the alert that carries real investigative value.

Alert 4: file deleted (rule 553, level 7)

The deletion alert records the last known state (size 88 bytes, SHA-256 `378189c1...831884`), which matches the hashes after the modification. This lets an analyst prove which version of the file was removed.

9.4 Framework and compliance mapping carried by the alerts

Rule	MITRE ATT&CK	Compliance tags
554 (added)	None assigned by this rule	PCI DSS 11.5, HIPAA 164.312.c.1 and c.2, NIST 800-53 SI.7, GDPR II_5.1.f, GPG13 4.11, TSC PI1.4/PI1.5/CC6.1/CC6.8/CC7.2/CC7.3
550 (modified)	T1565.001 Stored Data Manipulation (Impact)	Same tags as above
553 (deleted)	T1070.004 File Deletion (Defense Evasion); T1485 Data Destruction (Impact)	Same tags as above

9.5 Observed background alerts

Besides the test events, the endpoint produced registry FIM alerts under the default Windows configuration, for example rule 751 "Registry Value Entry Deleted" and rule 597 "Registry Key Entry Deleted" on keys beneath `HKLM\System\CurrentControlSet` at about 23:16 on 16 October. The rule 597 record I opened had `syscheck.mode: scheduled`, so it came from the periodic scan, not from my real-time directory. Its key was a Bluetooth pairing entry under `Services\BTHPORT` (the device address is omitted from this report for privacy). These alerts are discussed under Question 2.

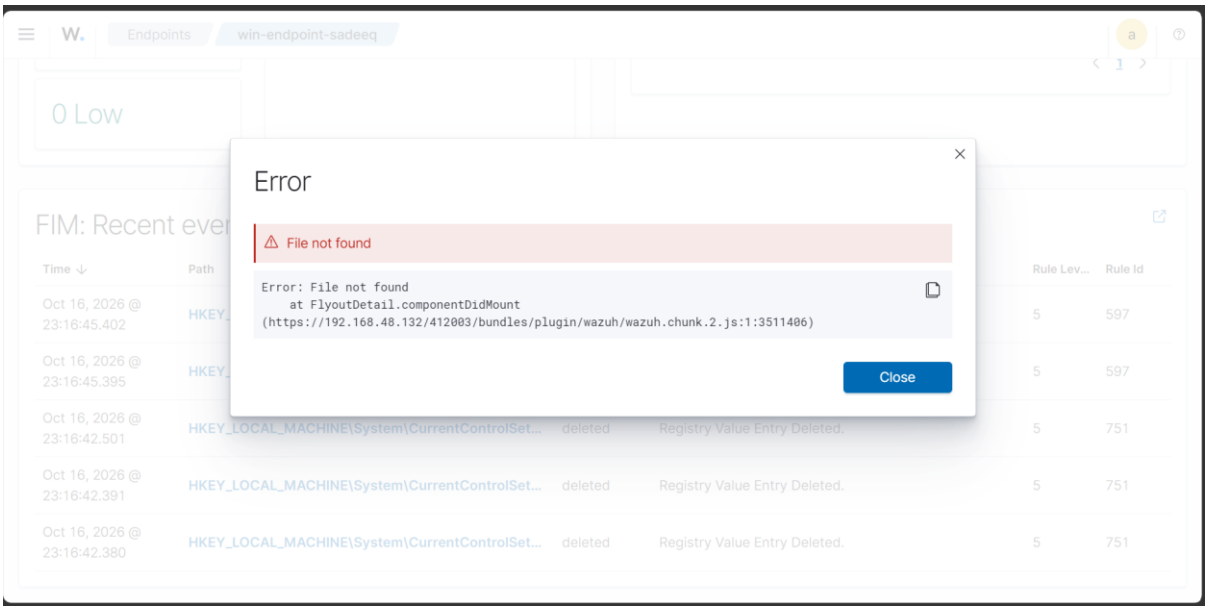


Figure 21: Default registry monitoring produced rule 751 and 597 alerts; opening a registry event raised a dashboard "File not found" flyout error

10. Responses to the four laboratory questions

Question 1: Alert triage (file modification alerts)

How to find them. In the agent's File Integrity Monitoring module, or in Discover, I filtered on `rule.id: 550` (or `syscheck.event: modified`) and the path `confidential_report`. Two alerts matched, and they mean different things:

- **09:27:01 (low risk).** `changed_attributes` was only `mtime`, the diff was "No content changes were found", and size and all three hashes were unchanged. The file was rewritten with identical content. Real-world causes are saving a file without editing it, a backup or antivirus touching it, or a sync client.
- **09:27:13 (true positive).** Size, mtime, MD5, SHA-1 and SHA-256 all changed, size grew from 45 to 88 bytes, and `syscheck.diff` shows the added line `Unauthorized modification during SOC test`. This is a genuine change to content.

Triage decision process. (1) Confirm that the path is really sensitive. (2) Check the time against approved change windows. (3) Compare before and after hashes and size to decide whether the content changed or only metadata. (4) Read the diff for what was added. (5) Look for neighbouring alerts on the same agent (added, deleted or many modified files) to see whether it is isolated or part of a pattern. (6) Escalate or close with a reason.

Limit of this evidence. `uname_after: Administrators` is the owner of the file, not the person or process that changed it. To identify the actor, FIM would have to be configured with `whodata="yes"` (Windows audit-based monitoring), which was not enabled in this lab.

Question 2: False positive identification

FIM produces false positives when it monitors locations where change is routine and harmless. I saw one example in this lab: the default Windows configuration also watches

registry keys, and the endpoint raised rule 751 and 597 alerts on keys under `HKLM\System\CurrentControlSet` (the rule 597 one was a Bluetooth pairing entry) even though nobody tampered with anything. Those alerts also carried alarming MITRE labels (T1070.004, T1485, T1112), which shows that a high label is not proof of an attack. My own overwrite at 09:27:01 is a small example of mtime-only noise.

Directories and keys that would normally generate excessive or unnecessary alerts, from general Windows practice (I did not monitor these in the lab):

- `C:\Windows\Temp` and user `%TEMP%` folders, browser caches and profiles: files appear and vanish constantly.
- The Recycle Bin and `C:\Windows\SoftwareDistribution` (Windows Update downloads).
- Application and system log folders and databases that are written continuously.
- Volatile registry paths such as `CurrentControlSet\Services` device and pairing entries.

Tuning. Narrow `<directories>` to what really matters and avoid recursive monitoring of busy paths; add `<ignore>` or `<ignore type="sregex">` entries for known-noisy files; use `<registry_ignore>` for noisy registry keys; use the `restrict` attribute to watch only certain file names; and raise or lower alert levels with local rules. For files where the content itself is confidential, use `<nodiff>` so that content is not copied into alerts.

Question 3: Ransomware detection

Ransomware shows up in FIM as a **burst** of events instead of a single one. The pattern to look for on one agent is: many rule 550 (checksum changed) alerts across many different files within seconds, every file's hash and size changing, new files with unusual extensions or ransom notes (rule 554), and original files being removed (rule 553). My three tests reproduce that pattern in miniature on one file (add, modify, delete), with each step appearing within about a second.

What makes this useful in practice:

- **Real-time mode.** The default scan interval here is 12 hours (43200 s). Ransomware would finish long before a scheduled scan noticed it, so real-time monitoring is essential on important folders.
- **Correlation.** A Wazuh frequency rule can raise a high-severity alert when many modifications hit one agent in a short window. An illustrative example (a recommendation only; I did not deploy or test it, and the thresholds would need tuning):

```
<group name="syscheck,ransomware,">
  <rule id="100100" level="12" frequency="20" timeframe="30">
    <if_matched_sid>550</if_matched_sid>
    <description>Possible ransomware: many files modified in 30
seconds</description>
  </rule>
</group>
```

- **Canary files and whodata.** Placing decoy files in monitored folders and enabling `whodata` gives early warning and identifies the responsible process.

- **Response.** Active Response can isolate the host or kill the process, and offline backups remain the real recovery control.

FIM only detects ransomware; it does not stop it. It should sit beside endpoint protection, backups and network segmentation.

Question 4: Centralised monitoring versus native Windows Event Logs

Windows Event Logs record events locally on each machine. Wazuh centralises and enriches them. The differences I could see in this lab:

Aspect	Native Windows Event Logs	Wazuh (as used in this lab)
Where evidence lives	On the endpoint; an attacker with administrator rights can clear or alter the logs	Alerts are forwarded to the manager within about a second and stored off the host (Figure 19)
File-change detail	Needs object auditing configured per folder; events show an access, not a before/after state	Before/after size, MD5, SHA-1, SHA-256, mtime and a content diff in one alert (Alert 3)
Context	Raw event IDs	Rule description, severity level, MITRE ATT&CK technique, PCI DSS / HIPAA / NIST / GDPR tags
Scale	Check each machine separately	One Dashboard for all agents, with search, filters and retention in the Indexer
Alerting and response	Manual or separate tooling	Rules, frequency correlation and Active Response built in

In this lab, one query in Discover returned all four events with their hashes and diff, which would have meant opening and correlating separate logs on the endpoint if I had relied on Windows Event Logs alone. Event Logs are still valuable as a source: Wazuh can collect them too, and the two are complementary.

11. Technical challenges and resolutions

These are the real problems I met, and how each was handled.

#	Problem	Resolution
1	In VMware, the new VM showed "Cannot connect the virtual device sata0:1" and the installer then stopped with "Unable to find a medium containing a live file system" (Figure 22, Figure 23).	The virtual CD/DVD drive was not attached to the ISO. I set CD/DVD to "Use ISO image file" with "Connect at power on" ticked, powered the VM off and on, and the Ubuntu installer started.
2	I forgot the password of my self-built Ubuntu VM. A GRUB recovery boot only produced a blank screen.	Nothing had been installed on that VM, so no work was lost. I used the ICDFA-provided server VM instead, which already contained the Wazuh components (Section 3).
3	The first Dashboard login attempt was rejected ("Invalid username or password").	A later attempt with the credentials supplied for the lab VM succeeded (Figure 7).

#	Problem	Resolution
4	The server's `ens33` was DOWN with no address, so the Dashboard was unreachable. It happened again later ("took too long to respond"), with `ens33` UP but no IPv4 (Figure 24, Figure 25).	`sudo ip link set ens33 up` and `sudo dhclient -v ens33` obtained 192.168.48.132 again. This fix was manual and temporary. I did not make it permanent (see recommendations).
5	`NET START WazuhSvc` returned "The service name is invalid" (Figure 26 and Figure 8).	The MSI was still installing in the background because of `/q`. After it finished, the same command started the service (Figure 9).
6	`sudo var/ossec/bin/wazuh-control info` returned "command not found".	I had omitted the leading `/`. I confirmed the file exists (`sudo ls -l /var/ossec/bin/wazuh-control`) and read the version from `dpkg -l | grep wazuh` (Figure 3).
7	I ran the file creation command twice, producing an extra alert.	Kept and documented as Alert 2 (mtime only). No alert was removed.
8	Opening a registry event in the Dashboard raised a "File not found" flyout error (Figure 21).	A Dashboard display bug for registry events. I used Discover and the raw alert JSON instead.
9	Agents 001 and 002 already existed on the manager.	I identified my agent by name (`win-endpoint-sadeeq`) and ID 003, and left the other records untouched.

Screenshots for the challenges

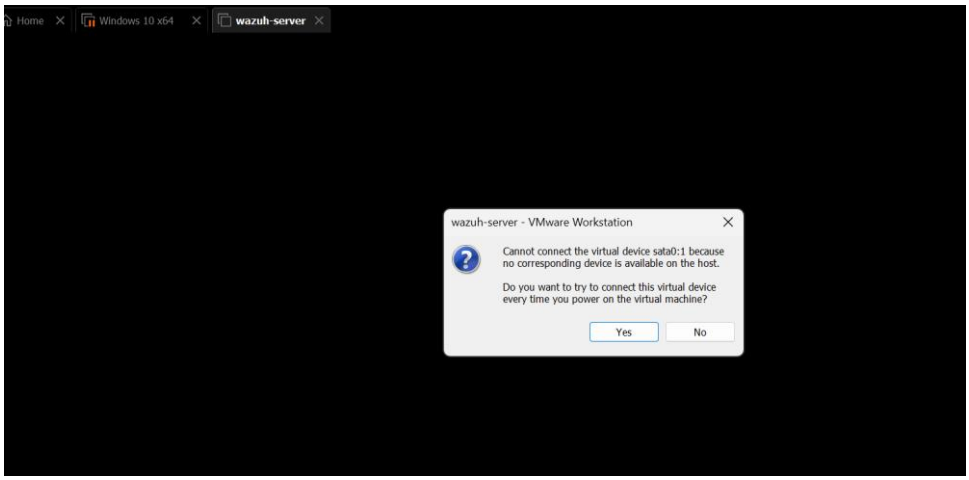


Figure 22: Challenge 1: virtual CD/DVD device error in VMware Workstation



Figure 23: Challenge 1: installer could not find the live file system

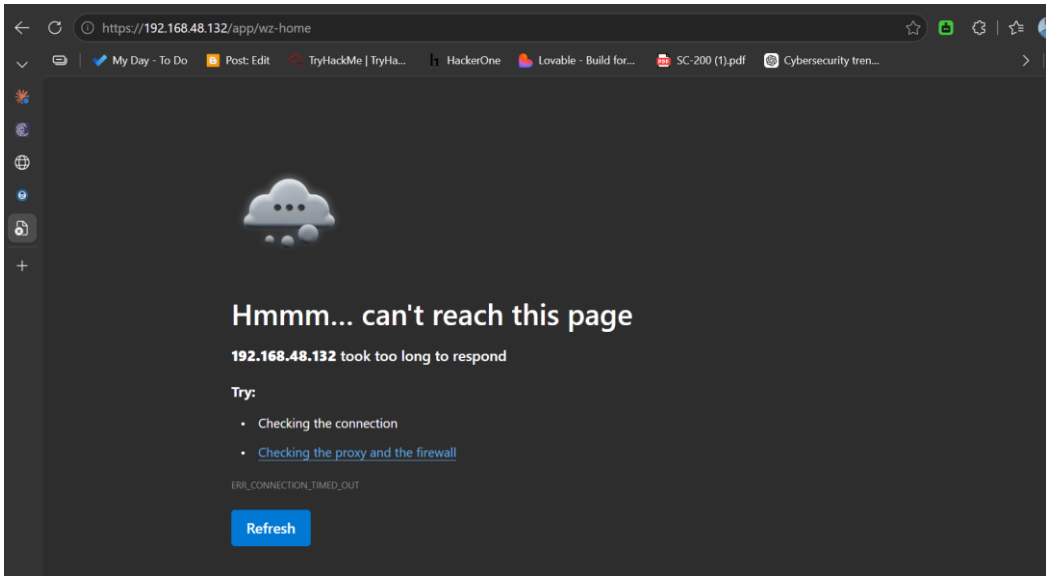


Figure 24: Challenge 4: Dashboard unreachable (ERR_CONNECTION_TIMED_OUT)

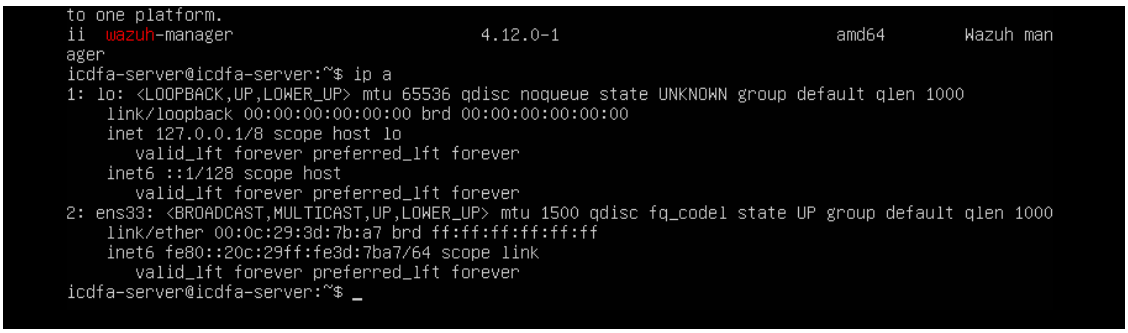


Figure 25: Challenge 4: ens33 UP with only an IPv6 link-local address

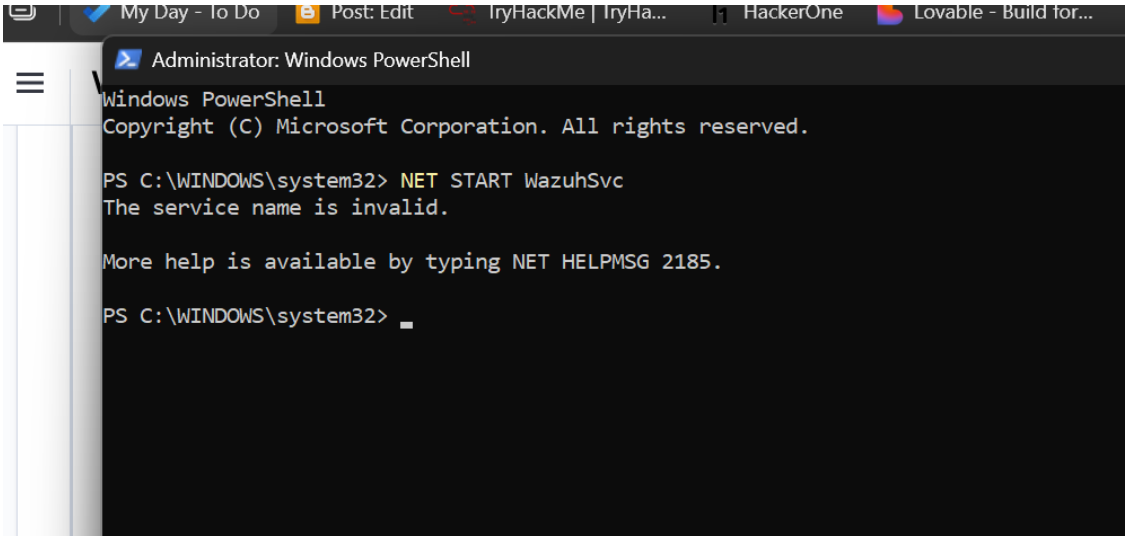


Figure 26: Challenge 5: NET START WazuhSvc before the agent finished installing

Limitations

- The endpoint was a single machine (my host Windows 11), so cross-host correlation was not demonstrated.
- `whodata` was not enabled, so the alerts do not name the user or process that made the change.
- The ransomware rule in Question 3 is an illustrative recommendation and was not tested.
- The Wazuh server software was provided pre-installed on the ICDFA VM and verified, not installed, by me.

12. Conclusion and recommendations

A centralised monitoring environment was verified and used end to end. Wazuh 4.12.0 (Manager, Indexer, Dashboard) was running on the Ubuntu server, a Windows 11 agent was registered and showed **active**, and real-time FIM on `C:\SensitiveFiles` was confirmed in `ossec.log`. Creating, modifying and deleting a test file produced four alerts (rules 554, 550, 550 and 553), each within about one second, with hashes, sizes, times and a content diff. The alerts answer the CISO's concern: unauthorised file changes on a workstation can be detected quickly, investigated with before/after evidence, and kept off the endpoint.

Recommendations

- **Make the server network persistent.** The address was lost twice. Configure `ens33` in `/etc/netplan` (or a DHCP reservation for the VM) so the manager keeps a stable address and agents do not disconnect.
- **Enable whodata** on sensitive directories to record which account and process made each change.

- **Tune FIM.** Keep monitoring focused, ignore known-noisy registry keys and temporary folders, and use `<nodiff>` for files whose content must not be copied into alerts.
- **Add correlation.** Deploy and tune a frequency rule for bursts of modifications, and consider Active Response, canary files and offline backups for ransomware resilience.
- **Maintain the platform.** Ubuntu 20.04 LTS has passed the end of its standard support (as stated in the VM login banner), so plan an upgrade or extended security maintenance, and keep agents at or below the manager version.
- **Housekeeping.** Review stale agent records 001 and 002 with the lab administrator before removing them.

13. References

[1] ICDFA Nigeria. Official practical brief and laboratory manual: Wazuh SIEM and File Integrity Monitoring laboratory (provided to students).

[2] Wazuh documentation. File integrity monitoring. https://documentation.wazuh.com/current/user-manual/capabilities/file-integrity/index.html

[3] Wazuh documentation. Installation guide. https://documentation.wazuh.com/current/installation-guide/index.html

[4] Wazuh documentation. Installing the Wazuh agent on Windows. https://documentation.wazuh.com/current/installation-guide/wazuh-agent/wazuh-agent-package-windows.html

[5] Wazuh documentation. Ruleset. https://documentation.wazuh.com/current/user-manual/ruleset/index.html

[6] MITRE ATT&CK. T1565.001 Stored Data Manipulation. https://attack.mitre.org/techniques/T1565/001/

[7] MITRE ATT&CK. T1070.004 Indicator Removal: File Deletion. https://attack.mitre.org/techniques/T1070/004/

[8] MITRE ATT&CK. T1485 Data Destruction. https://attack.mitre.org/techniques/T1485/

[9] MITRE ATT&CK. T1112 Modify Registry. https://attack.mitre.org/techniques/T1112/

Appendix A: Evidence checklist

Required evidence from the brief, mapped to the figures in this report and the files in the repository.

Required evidence	Where it is shown	Repository file
Technical report with cover page, introduction, objectives, architecture, deployment, configuration, results, alert analysis, four answers, challenges, conclusion and references	This document	report/

Required evidence	Where it is shown	Repository file
Successful installation of Wazuh components on the Ubuntu server	Figure 3, Figure 4, Figure 5	evidence/01 to 03
Successful login to the Wazuh Dashboard	Figure 7	evidence/06
Windows agent registered and Active	Figure 10, Figure 11	evidence/09, 10
SensitiveFiles directory configured for real-time FIM in ossec.conf	Figure 14, Figure 15	evidence/11, 13; config/
Test events and resulting alerts	Figure 17, Figure 18, Figure 19, Figure 20	evidence/16 to 18; alerts/

Appendix B: Command reference

On the Ubuntu server

```
dpkg -l | grep wazuh
sudo systemctl status wazuh-manager wazuh-indexer wazuh-dashboard --no-pager
ip a
sudo ip link set ens33 up
sudo dhclient -v ens33
```

On the Windows endpoint (Administrator PowerShell)

```
# agent install (generated by the Dashboard wizard)
Invoke-WebRequest -Uri https://packages.wazuh.com/4.x/windows/wazuh-agent-4.12.0-1.msi -
OutFile $env:tmp\wazuh-agent; msiexec.exe /i $env:tmp\wazuh-agent /q
WAZUH_MANAGER='192.168.48.132' WAZUH_AGENT_NAME='win-endpoint-sadeeq'
NET START WazuhSvc
New-Item -ItemType Directory -Path "C:\SensitiveFiles" -Force
Copy-Item "C:\Program Files (x86)\ossec-agent\ossec.conf" "C:\Program Files (x86)\ossec-
agent\ossec.conf.bak"
notepad "C:\Program Files (x86)\ossec-agent\ossec.conf"
Restart-Service WazuhSvc
Select-String -Path "C:\Program Files (x86)\ossec-agent\ossec.log" -Pattern
"sensitivefiles" -SimpleMatch | Select-Object -Last 5
Get-Date; Set-Content -Path "C:\SensitiveFiles\confidential_report.txt" -Value "Initial
confidential content - SOC lab test"
Get-Date; Add-Content -Path "C:\SensitiveFiles\confidential_report.txt" -Value
"Unauthorized modification during SOC test"
Get-Date; Remove-Item "C:\SensitiveFiles\confidential_report.txt"
```

In the Dashboard

```
Discover > index pattern wazuh-alerts-* > Last 24 hours > query: confidential_report
```